

# Informe de métricas: Puerta Inteligente para Mascotas con Control Adaptativo de Iluminación

Bianchi, Juan Alberto; Brocani, Agustín Nicolás; Ghiano, Gonzalo Agustín; Martucci, Federico Ariel; Minaudo, Lucas

DNI 30.475.902 - DNI 40.931.870 - DNI 43.448.036 - DNI 44.690.247 - DNI 43.316.993

Martes - Grupo M1

Universidad Nacional de La Matanza,  
Departamento de Ingeniería e Investigaciones Tecnológicas,  
Florencio Varela 1903 - San Justo, Argentina

## 1 Introducción

El objetivo de este informe es medir y promediar el uso de CPU (por core) y el uso de memoria (heap) en el microcontrolador ESP32, mediante la biblioteca Metrics.h provista por la cátedra, y documentar los resultados obtenidos en dos situaciones distintas de funcionamiento del sistema:

- Situación A: sistema en reposo (sin interacción del usuario durante 10 segundos).
- Situación B: sistema ejecutando una acción específica dentro de la ventana de medición.

Adicionalmente, cada situación se realiza sobre dos versiones del firmware:

- Versión sin vibecoding (implementación manual).
- Versión con vibecoding (implementación asistida por IA).

Para el sistema realizado con vibecoding se hizo uso de:

- Herramienta: Claude Code
- Modelo: Opus 4.8
- Esfuerzo: Max.

Uso de la biblioteca La biblioteca Metrics.h fue obtenida del repositorio oficial de la cátedra: <https://gitlab.com/so-unlam/Material-SOA/-/tree/master/Ejemplos%20SE/UsoCpuMemESP32>

La biblioteca se utiliza mediante dos primitivas principales:

- *initStats()*: se invoca dentro de setup() para iniciar el período de muestreo de CPU y memoria.
- *finishStats()*: se invoca cuando se desea cerrar la ventana de medición. La función imprime automáticamente por el Monitor Serial el promedio de uso de CPU (por core) y el promedio de heap libre durante el período muestreado.

## 2 Metodología

Para cada uno de los cuatro escenarios (A sin vibecoding, A con vibecoding, B sin vibecoding, B con vibecoding) se procedió de la siguiente manera:

- a. Se cargó el firmware al ESP32 mediante PlatformIO.
- b. Se abrió el Monitor Serial a 115200 baudios.
- c. Se ejecuta *initStats()* automáticamente al iniciar el sistema.
- d. Se dejó transcurrir la ventana de medición correspondiente a la situación (reposo de 10 s o acción específica).
- e. Se invocó *finishStats()* y se capturó la salida del Monitor Serial.
- f. Se registró la impresión resultante y se adjuntó como evidencia en este informe.

## 3 Desarrollo

### Situación A: Sistema en reposo

Descripción: dentro del loop() del firmware, tras 10 segundos sin interacción del usuario ni disparo de sensores, se invoca *finishStats()* y se registra la salida del Monitor Serial.

#### 3.1 Versión sin vibecoding

Captura del Monitor Serial:

```
=== Contribución Promedio al total del sistema ===
=== Tiempo de muestreado:      10(segundos) ===

=====
===== Estado Promedio del Uso de CPU en ESP32 =====
=====
Core 0 -> Total:  99.77% | Ocupado:   8.85% | Libre (IDLE):  90.92%
Core 1 -> Total:  98.17% | Ocupado:   1.49% | Libre (IDLE):  96.67%
36
=====
===== Estado Promedio de la memoria en ESP32 =====
===== Memoria interna (Heap) =====
=====
.Heap total : 325392 bytes
1Heap libre : 156424 bytes
9Heap usado : 168967 bytes

Uso          : 51.93 %

Muestreo de Metricas finalizado
```

**Observaciones:** Sistema prácticamente ocioso: la tarea IDLE (cuando un core no está haciendo nada útil) domina en los dos cores (>90%). El Core 0 aparece más cargado que el Core 1 (8,85 % vs 1,49 % ocupado), lo cual es esperable: en el ESP32 el Core 0 (PRO\_CPU) hostea el stack de WiFi/lwIP y, en esta versión, el TLS contra AWS IoT Core. El heap se mantiene estable en ~51,93 % (~169 KB usados de 325 KB). Todo consistente con tareas que duermen con *vTaskDelay* entre ciclos.

#### 3.2 Versión con vibecoding

Captura del Monitor Serial:

```
=== Contribución Promedio al total del sistema ===
=== Tiempo de muestreado:      10(segundos) ===

=====
===== Estado Promedio del Uso de CPU en ESP32 =====
=====
Core 0 -> Total:  98.77% | Ocupado:   9.82% | Libre (IDLE):  88.94%
Core 1 -> Total:  95.99% | Ocupado:  13.20% | Libre (IDLE):  82.80%

=====
===== Estado Promedio de la memoria en ESP32 =====
===== Memoria interna (Heap) =====
=====
Heap total : 330456 bytes
Heap libre : 188729 bytes
Heap usado : 141726 bytes
Uso        : 42.89 %

Muestreo de Metricas finalizado
```

**Observaciones:** También en reposo, pero con dos diferencias contra la versión sin vibecoding. En memoria usa bastante menos heap (42,89 %, ~142 KB vs ~169 KB; ~33 KB más libre). La diferencia se ve presente en las decisiones de arquitectura,

no por una “Optimización” de la IA en cuanto a recursos. Esta versión crea 4 tareas en vez de 6 ( $\approx 16$  KB menos en stacks, al no tener la tarea de luz ni la de telemetría separadas) y se conecta a HiveMQ por TCP plano, mientras que la de sin vibecoding usa AWS IoT Core con TLS + tres certificados + buffer MQTT de 1024 bytes, que se comen varios KB más. O sea, la comparación de memoria no es del todo pareja. En CPU la ocupación es un poco mayor y más ruidosa (Core 1 en 13,20 %), pero son números chicos dentro de la variación entre corridas.

## Situación B: Tras una acción específica

Descripción: entre *initStats()* y *finishStats()* se dispara una acción concreta del sistema.

En este caso se utilizó la siguiente acción: Se usó el sensor de proximidad y rfid, y permitiendo la apertura de la puerta tanto para adentro como para afuera..

### 3.3 Versión sin vibecoding

Captura del Monitor Serial:

```
=== Contribución Promedio al total del sistema ===
=== Tiempo de muestreado:      10(segundos) ===

=====
===== Estado Promedio del Uso de CPU en ESP32 =====
=====
5.23
Core 0 -> Total:  99.65% | Ocupado:    9.68% | Libre (IDLE):  89.97%
Core 1 -> Total:  97.83% | Ocupado:    1.72% | Libre (IDLE):  96.12%

=====
===== Estado Promedio de la memoria en ESP32 =====
===== Memoria interna (Heap) =====
=====
Heap total : 325392 bytes
Heap libre : 156295 bytes
Heap usado : 169096 bytes
Uso        : 51.97 %

Muestreo de Metricas finalizado
```

**Observaciones:** Los valores son prácticamente idénticos a los de reposo: el Core 0 pasa de 8,85 % a 9,68 % de ocupación y el Core 1 de 1,49 % a 1,72 %, con una variación de heap de apenas +129 bytes. La acción disparada no se refleja de forma apreciable en el promedio. Esto se debe a que la detección funciona por *polling* a cadencia fija: los sensores se leen en cada ciclo con o sin interacción, por lo que el costo de la acción puntual queda por debajo de la línea base del sondeo y del ruido de medición al promediar sobre la ventana de 10s.

### 3.4 Versión con vibecoding

Captura del Monitor Serial:

```
=== Contribución Promedio al total del sistema ===
=== Tiempo de muestreado:      10(segundos) ===

=====
===== Estado Promedio del Uso de CPU en ESP32 =====
=====
Core 0 -> Total:  99.24% | Ocupado:   4.81% | Libre (IDLE):  94.43%
Core 1 -> Total:  95.05% | Ocupado:   5.87% | Libre (IDLE):  89.17%

=====
===== Estado Promedio de la memoria en ESP32 =====
===== Memoria interna (Heap) =====
=====
Heap total : 330456 bytes
Heap libre : 188458 bytes
Heap usado  : 141997 bytes
Uso         : 42.97 %

Muestreo de Metricas finalizado
```

**Observaciones:** Los valores también resultan indistinguibles de los de reposo. La ocupación incluso disminuye respecto de la Situación A: el Core 0 pasa de 9,82 % a 4,81 % y el Core 1 de 13,20 % a 5,87 %, con una variación de heap de +271 bytes. Que la ocupación baje en lugar de subir confirma que el efecto de la acción se encuentra por debajo del piso de ruido de la medición, dominado por la variabilidad entre corridas del stack de red y las tareas de fondo. Al igual que en la versión sin vibecoding, la acción disparada no produce un impacto apreciable sobre las métricas promediadas.

## 4 Cuadro comparativo

| Métrica                   | A – sin IA | A – con IA | B – sin IA | B – con IA |
|---------------------------|------------|------------|------------|------------|
| CPU Core 0 – Ocupado      | 8,85 %     | 9,82 %     | 9,68 %     | 4,81 %     |
| CPU Core 0 – Libre (IDLE) | 90,92 %    | 88,94 %    | 89,97 %    | 94,43 %    |
| CPU Core 1 – Ocupado      | 1,49 %     | 13,20 %    | 1,72 %     | 5,87 %     |
| CPU Core 1 – Libre (IDLE) | 96,67 %    | 82,80 %    | 96,12 %    | 89,17 %    |
| Heap total                | 325 392 B  | 330 456 B  | 325 392 B  | 330 456 B  |
| Heap usado                | 168 967 B  | 141 726 B  | 169 096 B  | 141 997 B  |
| Heap libre                | 156 424 B  | 188 729 B  | 156 295 B  | 188 458 B  |
| Uso de heap               | 51,93 %    | 42,89 %    | 51,97 %    | 42,97 %    |

## 5 Conclusiones

Los resultados obtenidos en los cuatro escenarios revelan que el ESP32 funciona con una holgura significativa, registrando tiempos de inactividad (IDLE) superiores al 80 % en ambos núcleos. Este fenómeno demuestra que la organización mediante tareas y el uso de *vTaskDelay* para ceder el control del procesador resultan adecuados para la demanda actual, garantizando que ninguna implementación comprometa la capacidad de procesamiento del dispositivo.

Se observa como hallazgo central que el paso de la Situación A a la B no altera sustancialmente las métricas de rendimiento. En la variante manual, la carga de CPU oscila menos de un punto porcentual y el consumo de heap es casi idéntico; en la versión generada con IA, incluso se registra un descenso nominal de ocupación. Dicha estabilidad se atribuye al mecanismo de *polling* constante. En consecuencia, el costo marginal de una interacción puntual, es decir, encolar un evento, resolver la transición en la tabla de estados y accionar el servo— es de unos pocos microsegundos y resulta despreciable frente al trabajo de sondeo que la tarea de detección ejecuta en cada ciclo, con o sin interacción.

La discrepancia más notable se identifica en la comparativa entre firmwares, específicamente en el uso de recursos de memoria: la versión con vibecoding libera aproximadamente 27 KB adicionales de heap. No obstante, este ahorro no deriva de una mayor eficiencia de la IA, sino en diferencias en la arquitectura y los protocolos: la reducción en el número de tareas activas disminuye la reserva de *stacks*, mientras que el uso de TCP plano frente al cifrado TLS con certificados requerido por AWS IoT Core en la versión manual explica el resto de la diferencia.

## 6 Referencias

[1] Cátedra de Sistemas Operativos Avanzados, Universidad Nacional de La Matanza. "UsoCpuMemESP32 — biblioteca Metrics.h". Disponible en: <https://gitlab.com/so-unlam/Material-SOA/-/tree/master/Ejemplos%20SE/UsoCpuMemESP32>